

LAMP: ASP Rule Induction and Optimization with LLMs

David Bunker
University of Potsdam



Problem Setting

Writing ASP that is both correct and efficient is difficult. LAMP focuses on this challenge. To validate this via benchmarks, first a rule set from example facts is generated and the stable model it should produce.

$$AS(F \cup \Pi) = \{A^*\}$$

Here F is the given fact set, A^* is the target answer sets that can be generated by the Φ possible programs representing the Answer Set Programming program hypothesis space. The goal is to find a small program $\Pi \subseteq \Phi$ that reproduces A^* exactly.

Candidate programs use safe unary normal rules with bounded positive and negated body literals. Safe indicates every variable appears in a positive body literal, so grounding is finite. No new facts are allowed.

- Both LAMP and ILASP receive the same facts, target answer set and rule bounds.
- This makes the comparison about *search strategy*. LLM guided generation versus conflict-driven symbolic search.
- This task is intrinsically difficult. Bounded arity ungrounded normal ASP reasoning is Σ_2^P -complete and learning adds another search layer.

Example Benchmark

Facts
d0(o0). % vehicle(scooter).
d0(o1). % vehicle(car).
d1(o1). % slow_moving(scooter).

Target answer set
% { vehicle(scooter), vehicle(car), slow_moving(scooter), highway_possible(car) }
{ d0(o0), d0(o1), d1(o1), d2(o0) }

One minimal solution
% highway_possible(X) :- vehicle(X), not slow_moving(X).
d2(X) :- d0(X), not d1(X).

Correctness is judged by the stable model, not by reproducing the exact synthetic source rules. Many different rule sets can yield the same answer set.

Benchmark Design

Parameter	Values used
Predicates	6, 10
Terms	6, 10
Facts	5, 6, 9
Rules	11
Overall literals	2, 3 per rule body
Negative literals	0, 1, 2
Minimum derived atoms	1, 3

- 120 valid synthetic combinations were generated.
- Programs use unary predicates and normal rules only, which keeps the comparison controlled while still requiring nontrivial induction.
- Each benchmark has exactly one answer set and at least the indicated number of derived atoms.

LAMP System

- ASP is powerful for configuration, planning and combinatorial reasoning, but writing the rules is often the bottleneck.
- LAMP uses the LLM to generate ASP rules directly from facts, a target answer set and rule bounds.
- This differs from using LLMs mainly as a front end for natural-language translation or ILASP task construction.
- Every candidate is checked by CLINGO at inference time, without LLM fine-tuning.
- Because the task is rule induction rather than text translation, success is evidence of structured reasoning rather than prompt memorization.

ILASP Baseline

ILASP solves the same task under the Learning from Answer Sets framework, but does so with conflict driven symbolic search over an ASP metaprogram rather than iterative LLM generation.

- The hypothesis space is fixed by a mode bias derived from the benchmark profile.
- The target answer set is encoded as a single context-dependent positive example.
- In this study ILASP is the main symbolic baseline, run with a 420s timeout.

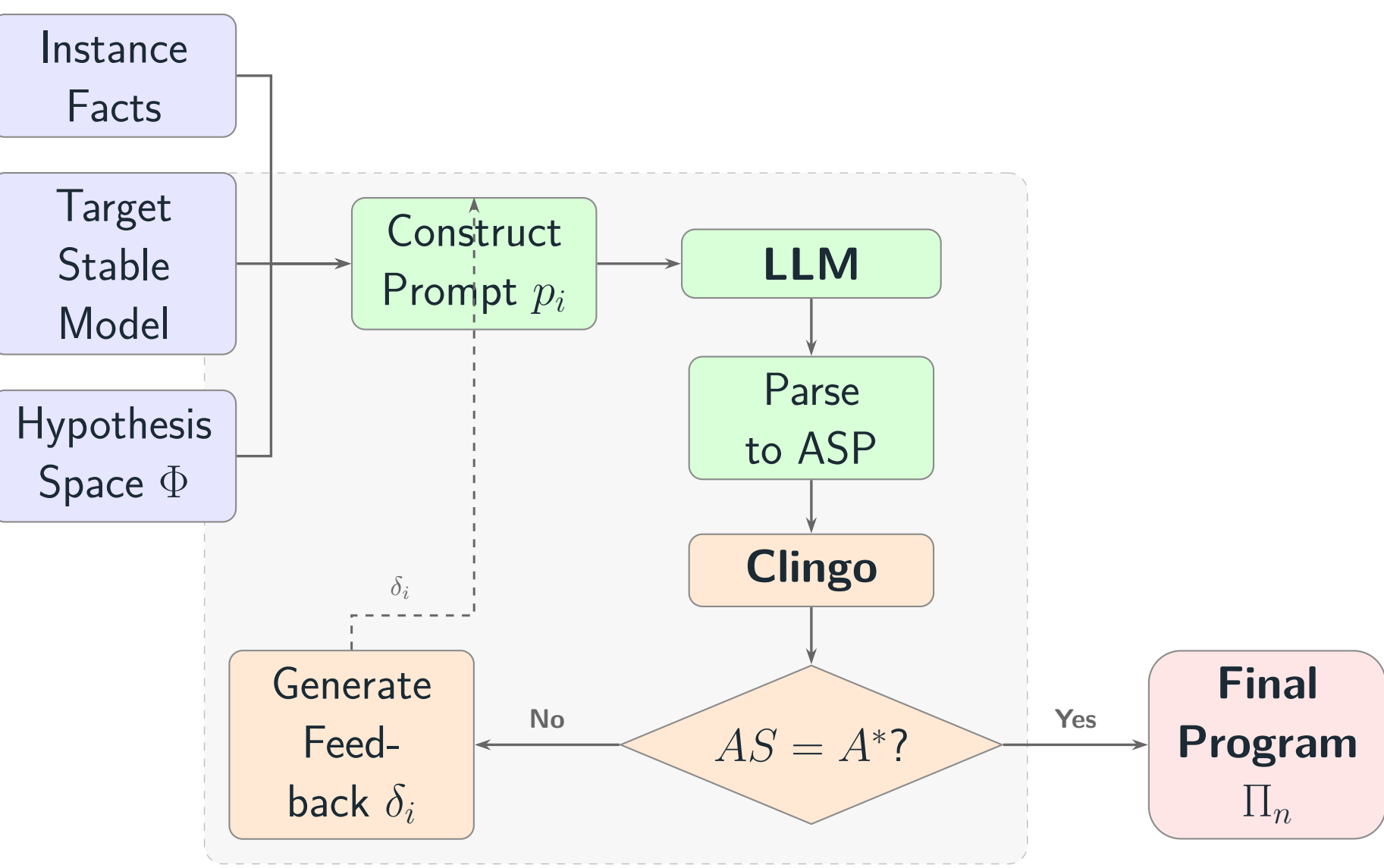
Experimental Controls

- Same facts, target answer set and bounded rule schema are given to both LAMP and ILASP.
- Correctness is exact CLINGO stable model matching and not surface similarity to the original rules.
- LAMP used 3 attempts and ILASP and LAMP used the same 420s timeout.
- Each model was run once per benchmark at temperature 1, so small score gaps should be interpreted cautiously.

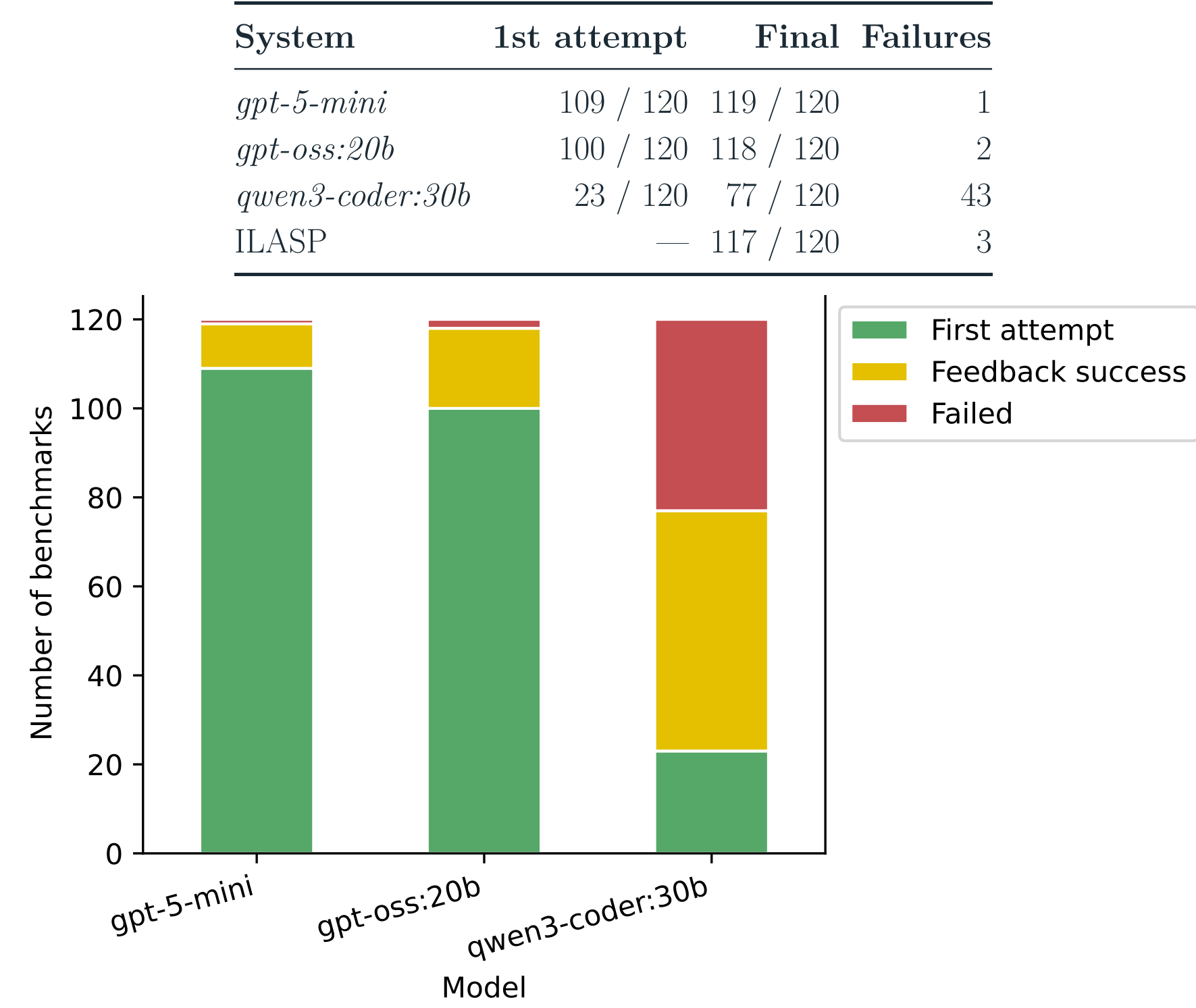
Source: <https://github.com/dbunker/lamp>

LAMP Protocol

- Prompt the LLM with instance facts F , target answer set A^* , rule schema bounds and a worked example.
- Parse the raw output into an ASP candidate program Π_i .
- Run $F \cup \Pi_i$ with CLINGO.
- If the produced answer set mismatches A^* , feed the mismatch back and ask for a corrected rule set.
- Stop after success or after 3 total attempts.

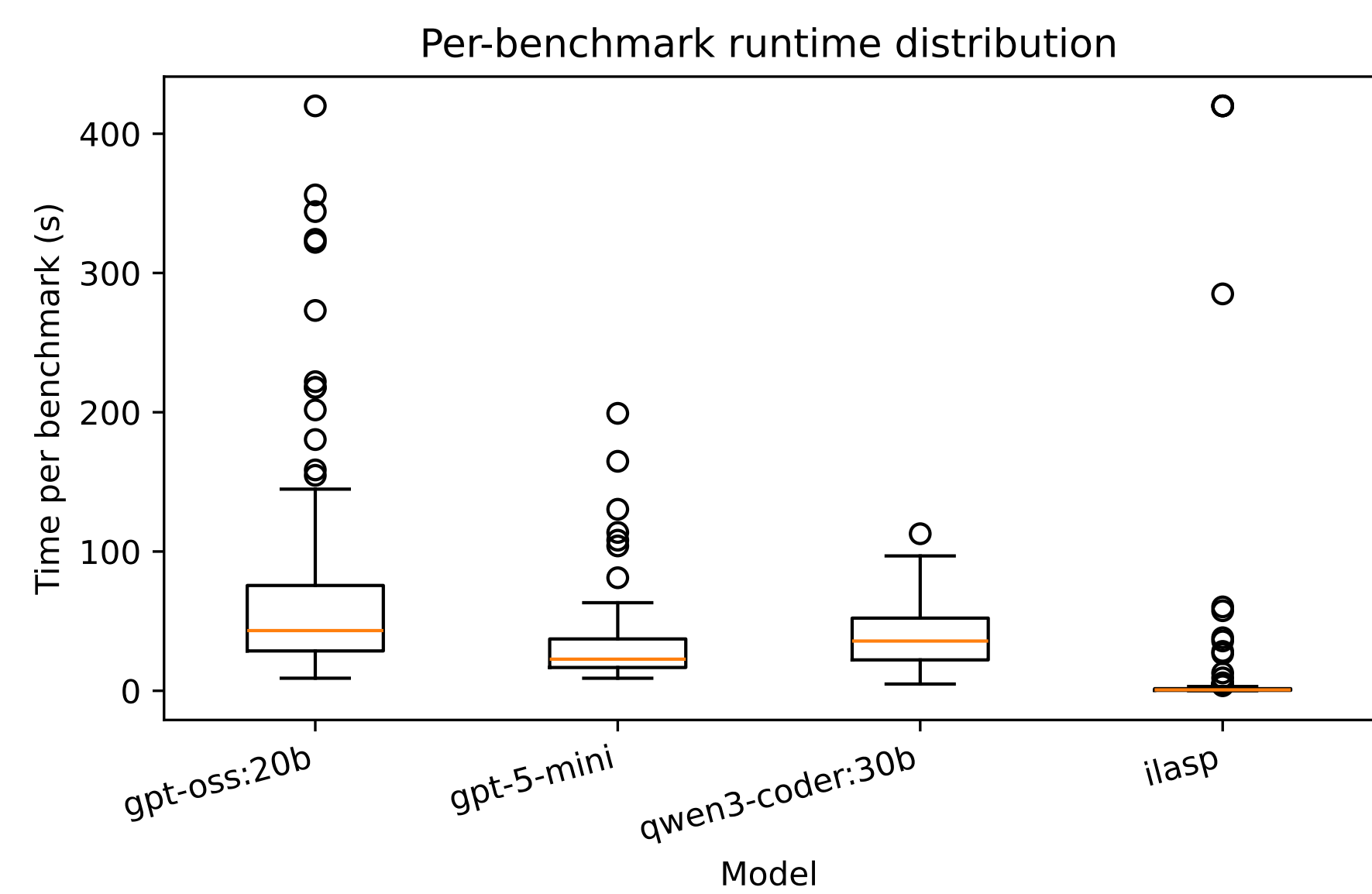


Accuracy Gains From Feedback



Feedback recovered 10 extra cases for *gpt-5-mini*, 18 for *gpt-oss:20b* and 54 for *qwen3-coder:30b*. The iterations improve the rules when the first draft is close but not exact.

Runtime And Search Behavior



System	Total	Mean	Median	Max
gpt-5-mini	3805.46	31.71	22.70	199.22
gpt-oss:20b	8630.91	71.92	43.22	420.00
qwen3-coder	4577.38	38.14	35.72	112.77
ILASP	1917.36	15.98	0.66	420.00

Times are seconds. LLM totals accumulate all feedback iterations. ILASP timeouts are recorded as 420s. *gpt-5-mini* timing excludes network latency.

Program Quality

Mean output literal count $L(\Pi)$ for correct solutions				
Original	gpt-5	gpt-oss	qwen	ILASP
22 literals	4.81	5.15	5.85	3.67
33 literals	3.49	3.82	3.84	2.71

Stratified solved programs	
System	Stratified / Solved
Original benchmarks	49 / 120
gpt-5-mini	119 / 119
gpt-oss:20b	117 / 118
qwen3-coder:30b	77 / 77
ILASP	117 / 117

Both ILASP and strong LAMP models compressed the synthetic source programs heavily. Correct outputs usually needed only a handful of literals and almost every solved program became stratified even when the original benchmark was not.

Timeout Cases

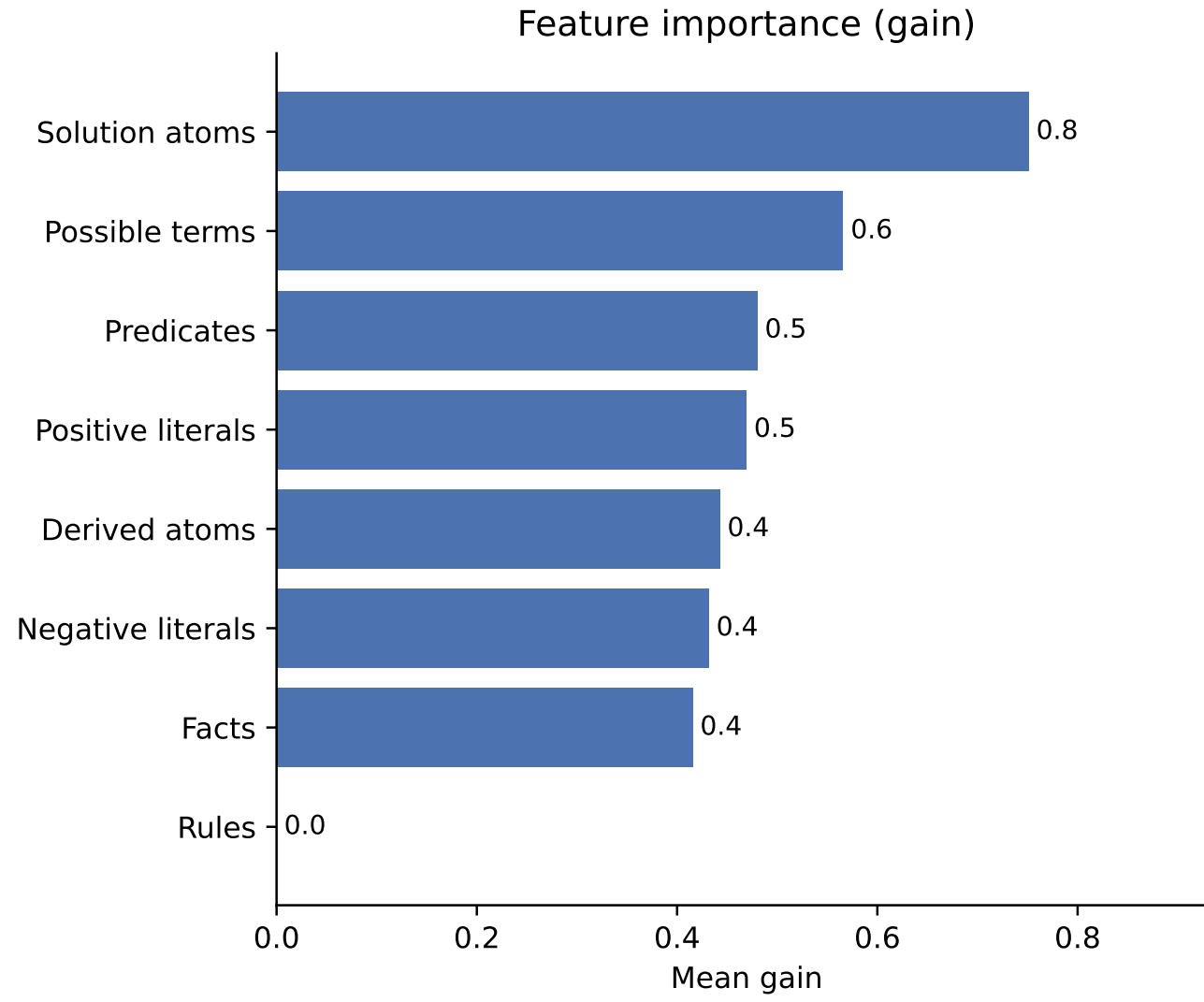
Benchmark	gpt-5	gpt-oss	qwen	ILASP
7	✓	✓	×	420 s
27	✓	✓	×	420 s
67	✓	✓	×	420 s

Structural profile of ILASP timeout cases			
Benchmark	Grounded	Hyp. space	Neg. lits.
7	27	19906	11
27	20	16960	11
67	27	2206	11
Dataset avg	13.28	3255.22	8.80

These were timeouts, not logical contradictions. Benchmark 67 shows that grounded rules and negation can still cause search cliffs even when the hypothesis space is smaller.

Failure Corollaries

LAMP can also be used to indicate which features are associated with LLM failure. The strongest models missed too few cases for a useful classifier, so the analysis focuses on *qwen3-coder:30b*.



The top gain features describe scale and vocabulary breadth. Larger target answer sets, more possible terms and more predicates make exact rule recovery harder. XGBoost reached 61.7% cross-validation accuracy with AUC 0.619, below the 64.2% majority baseline, so these are descriptive correlates rather than reliable predictors.

Research Questions

- RQ1:** Solver-in-the-loop prompting can produce correct ASP rules from facts and a target answer set.
- RQ2:** LAMP nearly matches ILASP in accuracy. ILASP is faster on average, but LAMP solves the ILASP timeout cases.
- RQ3:** Failure correlates with target size and vocabulary breadth, but the current sample is likely too small for reliable prediction.

Scope And Caveats

- The study is synthetic and restricted to unary predicates, normal rules and exactly one answer set.
- gpt-5-mini* timing excludes network latency; ILASP failures are recorded at the 420s timeout ceiling.
- Richer ASP constructs, ASPBench, planning domains, repeated trials and prompt optimization would strengthen the evaluation.
- LAMP is not a replacement for ILASP, but could be complementary for an LLM guided search strategy.

Key Results

Across 120 synthetic rule induction benchmarks, the strongest LAMP models nearly matched ILASP, solved every ILASP timeout case and usually produced smaller, more stratified programs than the synthetic originals.

99.2%

Best final accuracy with LAMP
gpt-5-mini: 119 / 120

98.3%

Best local model result
gpt-oss:20b: 118 / 120

3 / 3

ILASP timeout benchmarks solved by both strong LAMP models

3-6

Average output literals for correct solutions vs. 22 or 33 originals

Hybrid Symbolic LLM Search

What ILASP contributes

- Usually subsecond on this benchmark suite.
- Conflict driven search over a principled mode bias.
- Very concise programs within the bounded hypothesis space.

What LAMP contributes

- Token generation explores the rule space differently.
- gpt-5-mini* and *gpt-oss:20b* solved all 3 ILASP timeout cases.
- Every proposal is still solver-checked by CLINGO.

Natural hybrid

- Use ILASP search as the reliable symbolic core.
- When search stalls, ask an LLM for candidate rules or refinements.
- Accept only candidates that pass exact stable-model verification.

ILASP searches systematically ↔ LLM proposes when search stalls ↔ Clingo accepts only exact stable-model matches